

Link Prediction

Supervised vs Unsupervised

The supervised learner doesn't know much



The unsupervised learner knows what it is doing



Supervised learning tasks with networks

- **Node classification**

- Given a network (e.g. friendship network) and some labels (e.g. political party). Can we predict the labels of a node from the labels of their neighbours?

- **Graph classification**

- Given many networks (e.g. ego-networks, brain networks) and outcomes (e.g. political party, mental disorders). Can we predict the outcomes from the topology of the network?

- **Link prediction**

- Given a network (e.g. friendship network) and optionally some metadata (e.g. political party). Can we predict which links we are missing (or will be created)?

Link prediction



Does this link exist?

Link prediction



Does this link exist?

Many tasks

1. Model Validation

- Observe part of the adjacency matrix (fit model)
- Predict held out entries (cross validation)

Link prediction



Does this link exist?

Many tasks

1. Model Validation

2. De-noising / network reconstruction

- Real-world data are noisy / contain errors

Link prediction



Does this link exist?

Many tasks

1. Model Validation

2. De-noising / network reconstruction

3. Predict missing links

- Observed edges are assumed correct
- Predict which unobserved edges exist

Link prediction



Does this link exist?

Many tasks

1. Model Validation

2. De-noising / network reconstruction

3. Predict missing links

4. Predict future links

- Observe the adjacency matrix at time (t)
- Predict edges in time $(t+1)$

Link prediction



Does this link exist?

Many tasks

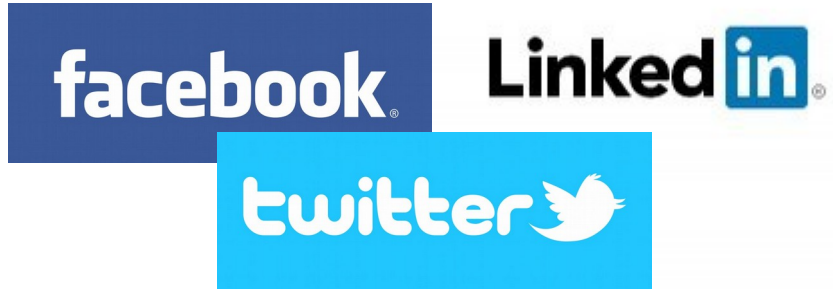
1. Model Validation

2. De-noising / network reconstruction

3. Predict missing links

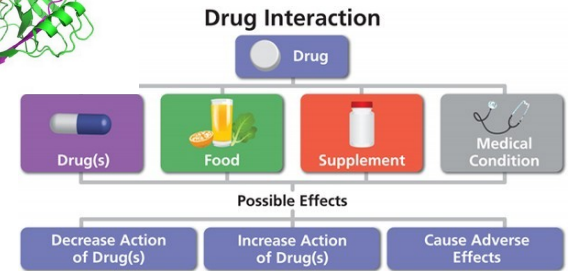
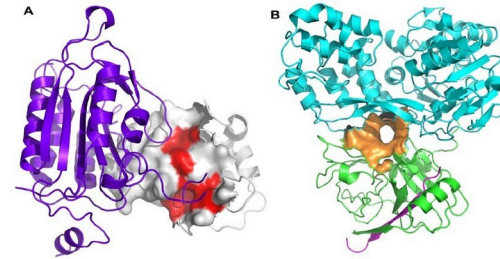
4. Predict future links

Applications

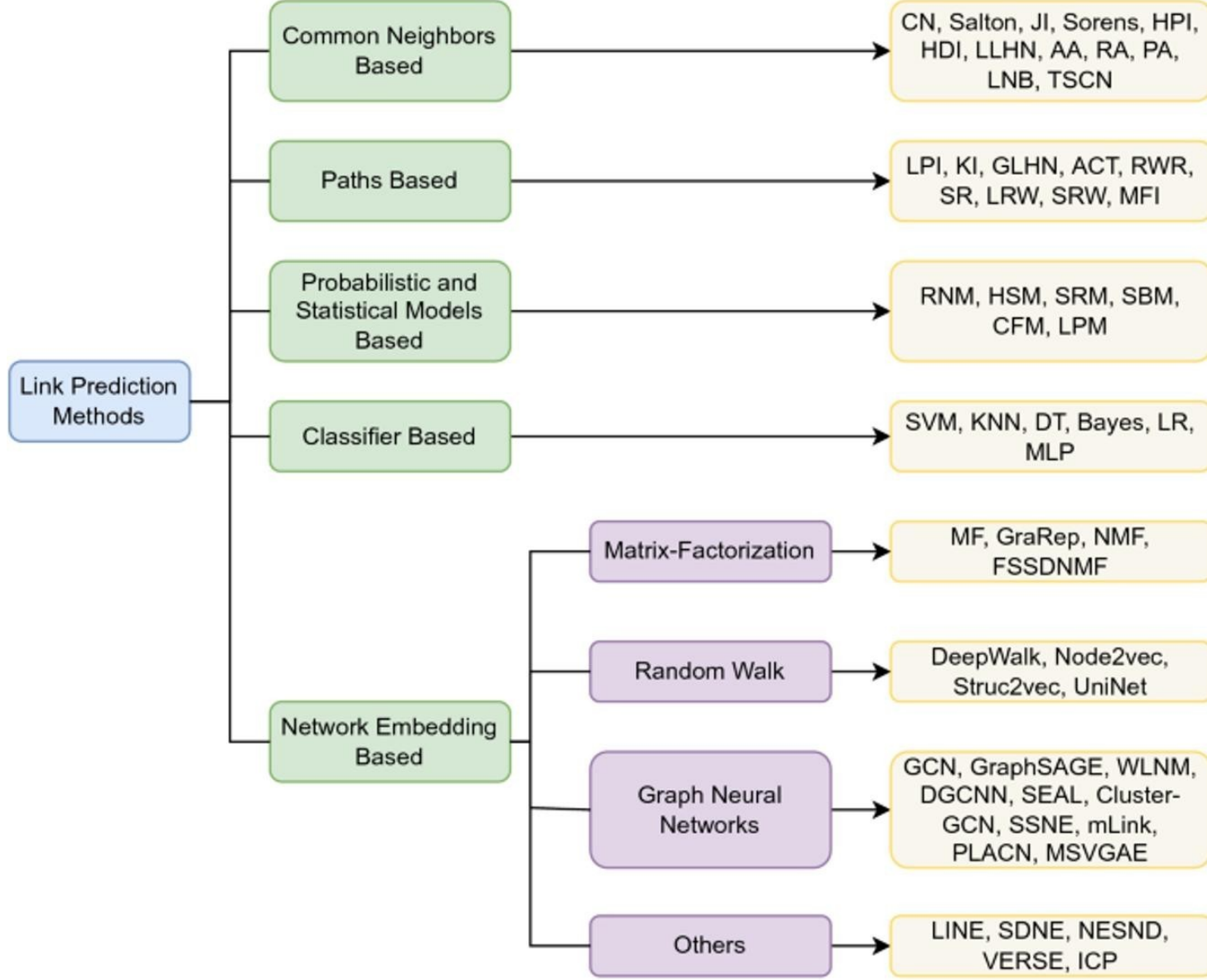


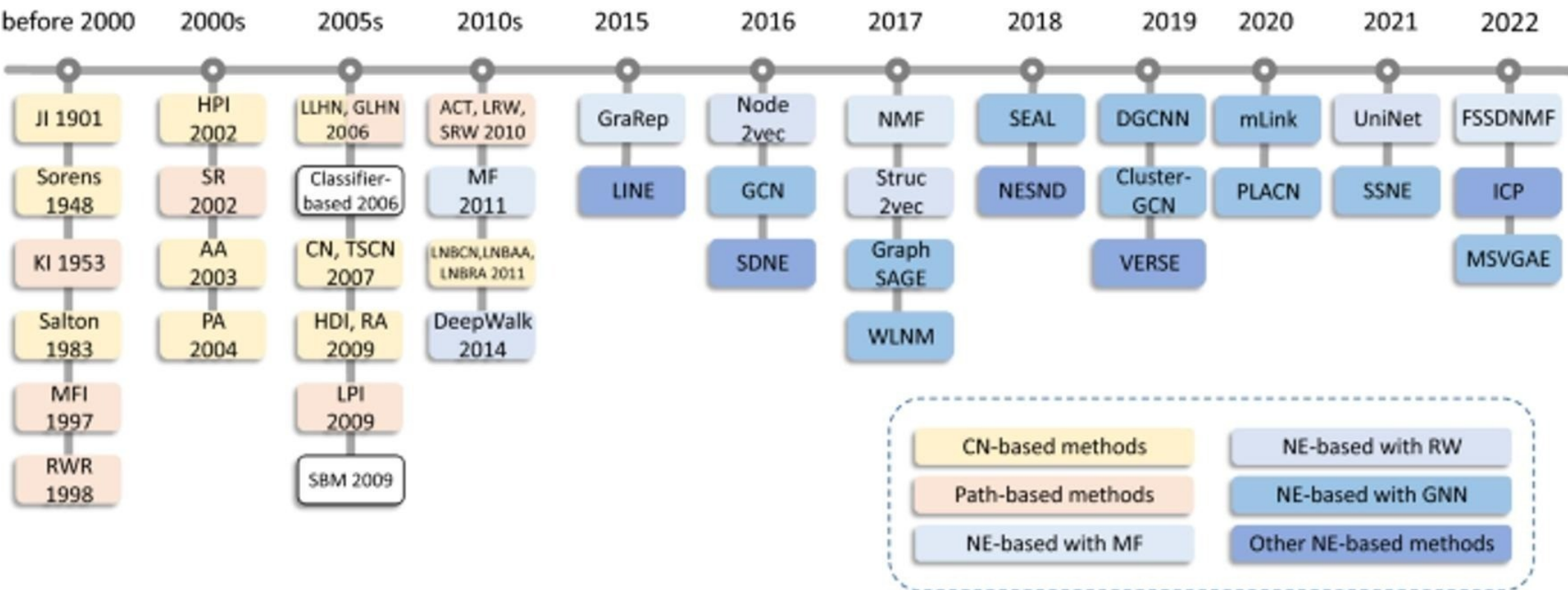
Suggesting social and professional connections

Predicting biological interactions



Recommending products and services





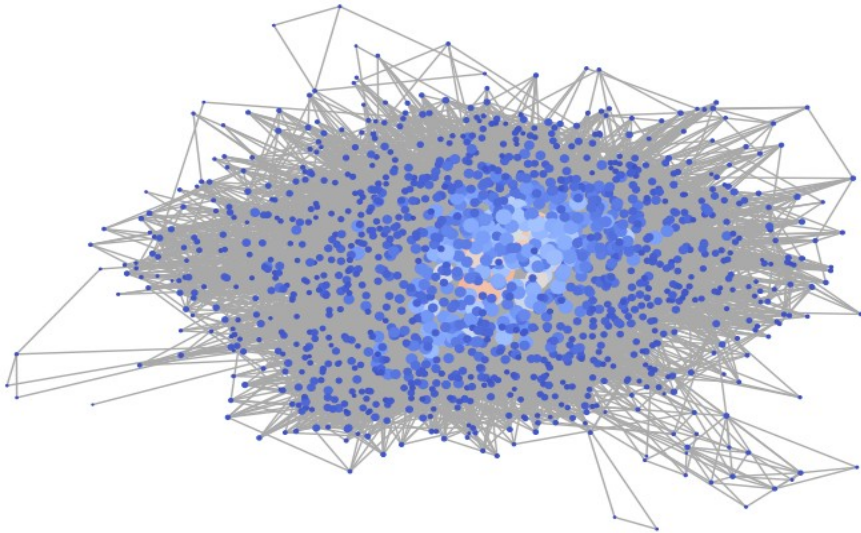
Predicting missing links

Goal: Rank all non-edges according to how likely they are to exist

Assessed using measures such as accuracy, F1, AUC...

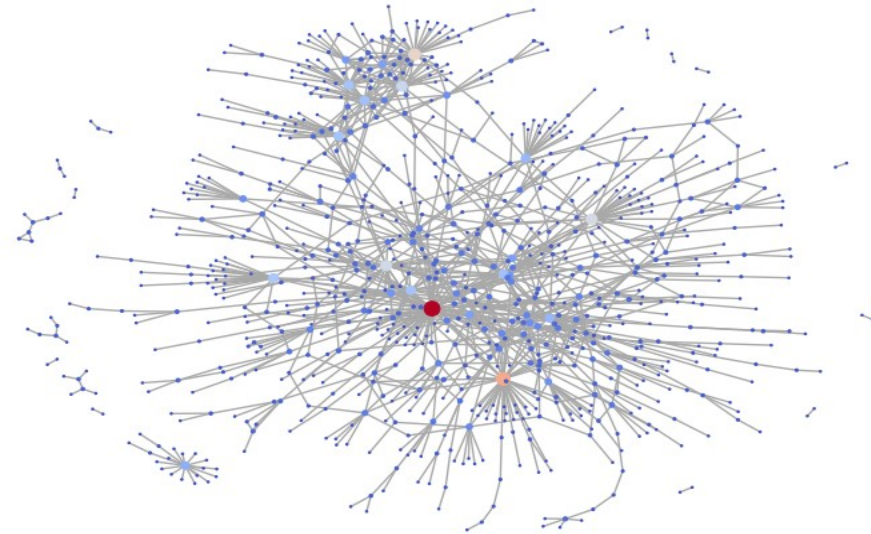
Data Challenge: Link prediction

Twitter network

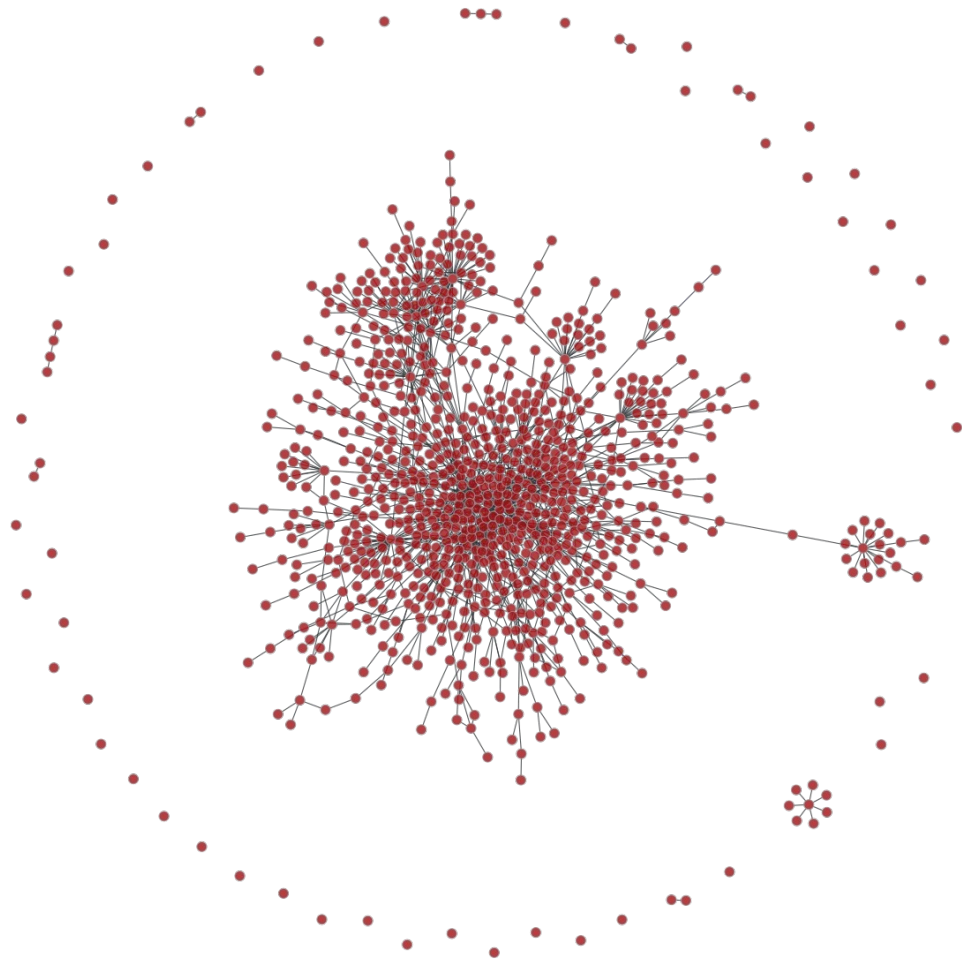


Global clustering: 0.172
Degree assortativity: -0.033

PPI network



Global clustering: 0.014
Degree assortativity: -0.157



Protein-protein interaction network in *S. cerevisiae*
Clustering ~ 0 ; Assortativity ~ -0.2

We have removed some edges, your objective is to predict those accurately.

We give you:

- Graph: Used for training
- Test dataset (a series of node pairs, some with a link associated)

How:

- Methods based on common neighbors
- Methods based on paths
- Methods based on embeddings
 - Spectral methods
 - Matrix factorization
 - Node2vec
 - GraphSAGE

Why not just use accuracy?

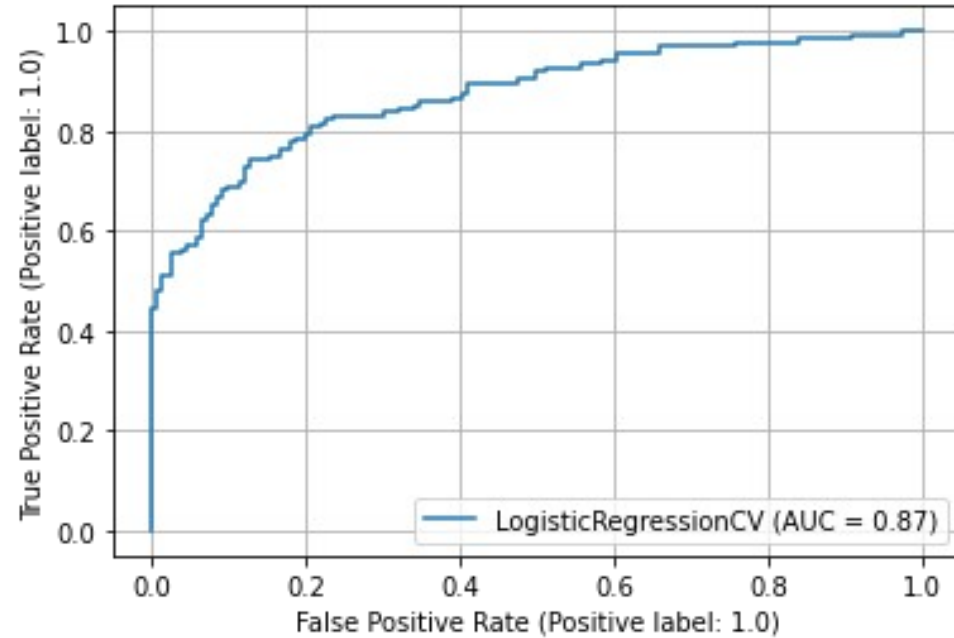
Possible non-edges vastly outnumber actual edges in real networks

Predicting “no edge” everywhere is close to 100% accurate — but useless

AUC-ROC ranks true edges above non-edges, regardless of threshold — robust to class imbalance

ROC curve

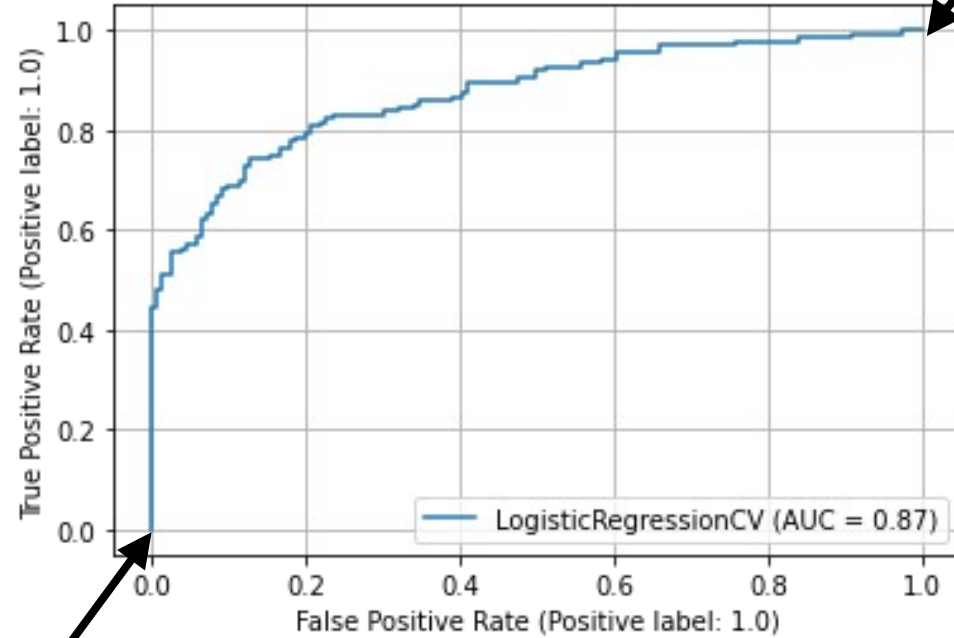
Edges predicted correctly
No. of edges



Non-edges predicted as edges
No. of non-edges

ROC curve

(everything predicted as an edge)



Edges predicted correctly
No. of edges

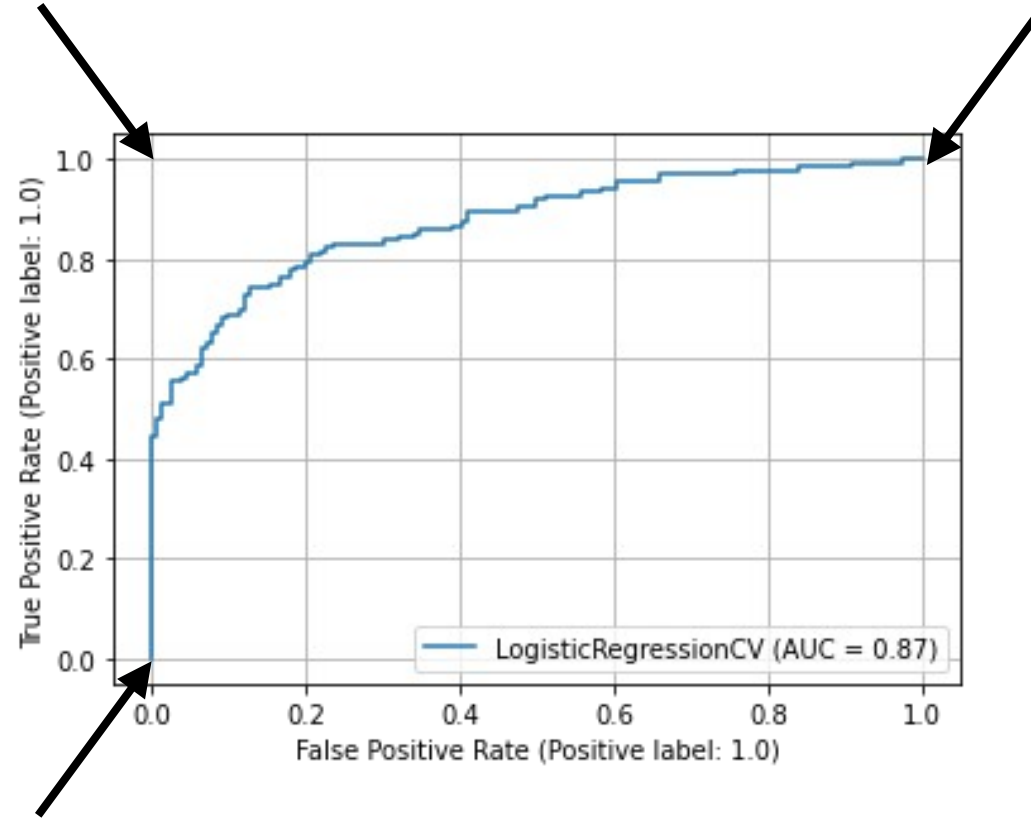
(nothing predicted as an edge)

Non-edges predicted as edges
No. of non-edges

ROC curve

(all edges correctly predicted)

(everything predicted as an edge)



Edges predicted correctly
No. of edges

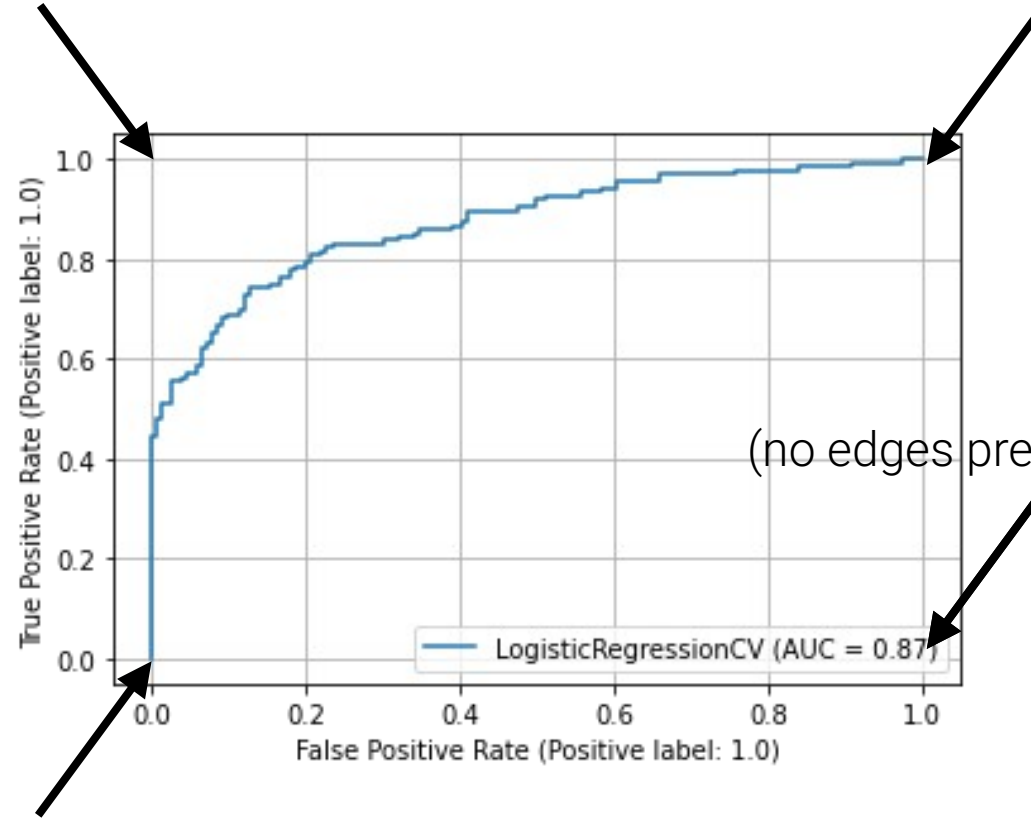
(nothing predicted as an edge)

Non-edges predicted as edges
No. of non-edges

ROC curve

(all edges correctly predicted)

(everything predicted as an edge)



(no edges predicted correctly)

Edges predicted correctly
No. of edges

(nothing predicted as an edge)

Non-edges predicted as edges
No. of non-edges

Sampling negatives

Non-edges vastly outnumber edges — scoring every pair is often infeasible

Standard practice: sample a set of non-edges equal in size to the held-out positive edges

AUC is then relative to this specific sample, not the full space of non-edges

The node holdout problem

Standard supervised learning holds out independent examples

Link prediction can't hold out nodes — a node's "features" are its neighbourhood, removed along with the node

Instead we hold out edges: both endpoints stay in the graph, just with fewer connections

An inherent limitation of link prediction, not a flaw in any one method

Practical Intro

Local heuristics (common neighbours approach)

Based on similarity of node connections

$\Gamma(x)$ $\leftarrow$ neighbours of x

k_x $\leftarrow$ degree of x

Common neighbours

$$s_{xy}^{\text{CN}} = |\Gamma(x) \cap \Gamma(y)|,$$

Jaccard similarity

$$s_{xy}^{\text{Jaccard}} = \frac{|\Gamma(x) \cap \Gamma(y)|}{|\Gamma(x) \cup \Gamma(y)|}$$

Cosine similarity

$$s_{xy}^{\text{Salton}} = \frac{|\Gamma(x) \cap \Gamma(y)|}{\sqrt{k_x \times k_y}}$$

k_x ← degree of x

Common neighbour heuristics

$\Gamma(x)$ ← neighbours of x

$\Gamma(x)$	$\Gamma(y)$	CN	Jaccard	Cosine
ABC	BC	2	0.66	0.81
ABC	BCD	2	0.5	0.66
ABC	C	1	0.33	0.57
ABC	CD	1	0.25	0.41
ABC	CDE	1	0.2	0.33

Common Neighbours ignores degrees

Jaccard and Cosine provide similar rankings

Practical: Local heuristics

$\Gamma(x)$	$\Gamma(y)$	CN	Jaccard	Cosine
ABCDEF	DEFGH	3	0.38	0.55
ABCDEF	DE	2	0.33	0.58

Jaccard and Cosine do not always provide the same ranking!

Jaccard is biased towards nodes with similar degree